

Dicas atuais para o Delphi: Soluções modernas para problemas comuns

Mário Leite

O **Delphi** continua sendo uma das ferramentas **RAD (Rapid Application Development)** mais produtivas do mercado, agora multiplataforma Windows®, macOS, iOS, Android e Linux) com o FireMonkey (FMX). Portanto, muita coisa mudou desde as versões clássicas (7, 2007, XE), mas a essência permanece: componentes, orientação a objetos e compilação nativa. Neste artigo, apresento **oito dicas** – cinco revisadas e atualizadas das originais, mais três novas - que vão ajudar você a escrever código mais limpo -, evitar dores de cabeça e aproveitar melhor os recursos do Delphi moderno (versão 12 Athens).

Dica 1 - Impedir maximização e duplo clique na barra de título (VCL)

Um problema comum: o usuário dá duplo clique na barra de título e o formulário maximiza, desfazendo todo o layout pensado para a tela. No VCL, isso pode ser evitado de duas formas:

Forma simples (sem código):

Na propriedade `BorderIcons` do formulário, remova `biMaximize`. O botão de maximizar desaparece e o duplo clique perde o efeito.

Forma programática (quando você precisa manter o botão, mas impedir a ação):
Capture a mensagem `WM_SYSCOMMAND`:

```
type
  TForm1 = class(TForm)
    procedure WMSysCommand(var Msg: TWMSysCommand); message WM_SYSCOMMAND;
  end;

implementation

const
  SC_TITLEDBLCLICK = 61490;

procedure TForm1.WMSysCommand(var Msg: TWMSysCommand);
begin
  case Msg.CmdType of
    SC_MAXIMIZE, SC_TITLEDBLCLICK: Exit;
  end;
  inherited;
end;
```

No **FireMonkey (FMX)**: a abordagem é diferente. Use o evento **OnResize** ou a propriedade `WindowState`. Se quiser impedir maximização, defina `BorderStyle` como `bsSizeable` e gerencie o estado desejado.

Dica 2 - Evitar que o protetor de tela (ou suspensão) atrapalhe processos longos

No passado, desabilitávamos o screen saver com `WM_SYSCOMMAND` e `SC_SCREENSAVE`. Hoje, o sistema operacional raramente usa animações de protetor de tela, mas entra em **suspensão** ou desliga o monitor – o que pode interromper um processamento demorado.

Solução moderna (Windows): use a API `SetThreadExecutionState` para informar ao sistema que a aplicação está ativa.

```

uses
    Winapi.Windows;

procedure ManterSistemaAtivo(Ativo: Boolean);
begin
    if Ativo then
        SetThreadExecutionState(ES_CONTINUOUS or ES_SYSTEM_REQUIRED or
            ES_DISPLAY_REQUIRED)
    else
        SetThreadExecutionState(ES_CONTINUOUS);
end;

// Uso:
ManterSistemaAtivo(True);    // antes do processamento longo
// ... processamento ...
ManterSistemaAtivo(False);   // ao terminar

```

Chame **ManterSistemaAtivo(True)** antes de iniciar um processamento longo e **ManterSistemaAtivo(False)** ao terminar.

No **FireMonkey multiplataforma**, não há uma API única - você pode usar threads para não travar a interface - mas a suspensão do dispositivo é controlada pelo sistema operacional de cada plataforma.

Dica 3 - Bloquear Alt+Tab e Ctrl+Alt+Del (alerta de segurança)

A técnica original usava `SystemParametersInfo(SPI_SCREENSAVER_RUNNING, ...)` e funcionava no Windows 9x/XP. **Nas versões modernas do Windows (7, 8, 10, 11), isso não funciona mais** – por razões de segurança, não se pode bloquear Ctrl+Alt+Del. O Alt+Tab pode ser bloqueado com hooks globais (ex.: `SetWindowsHookEx`), mas é uma prática **desaconselhada**, pois retira o controle do usuário e pode causar instabilidade.

Recomendação:

- Para aplicações *quiosque* (terminais de autoatendimento), use as configurações do Windows (políticas de grupo, shell customizado) em vez de código.
- Se mesmo assim precisar bloquear teclas, limite-se a capturar `WM_SYSKEYDOWN` para teclas específicas – mas isso só funciona enquanto seu aplicativo está em foco.

Esta dica está obsoleta e não deve ser usada em aplicações comuns. Remova-a de seu código legado.

Dica 4 - Fixar colunas no DBGrid (VCL)

No VCL, a propriedade `FixedCols` do `TDBGrid` é protegida, mas podemos acessá-la via *class casting*:

```

type
  TDBGridEx = class(TDBGrid);    // declaração na cláusula type

procedure TForm1.Button1Click(Sender: TObject);
begin
  (DBGrid1 as TDBGridEx).FixedCols := 1;    // fixa a primeira coluna (a da esquerda)
end;

```

No FireMonkey (TGrid), não há um recurso nativo de colunas fixas. Você pode simular congelando colunas com `ColumnWidth` e tratando o evento `OnDrawColumnCell` – ou migrar para componentes de terceiros (ex.: TMS FMX Grid).

Dica 5 - Descobrir a linha e coluna atuais em um DBGrid (VCL)

Também por meio de class casting:

```

type
  TDBGridEx = class(TDBGrid);

procedure TForm1.AppIdle(Sender: TObject; var Done: Boolean);
begin
  Caption := Format('Linha: %d Coluna: %d',
    [TDBGridEx(DBGrid1).Row, TDBGridEx(DBGrid1).Col]);
end;

procedure TForm1.FormCreate(Sender: TObject);
begin
  Application.OnIdle := AppIdle;
end;

```

Alternativa moderna: use o evento `OnCellClick` ou `OnDrawColumnCell` para obter as coordenadas no momento da interação, sem precisar de `Application.OnIdle`.

Dica 6 - Lidar com High DPI e monitores 4K (nova)

Para que sua aplicação não apareça “minúscula” ou “borrada” em telas de alta resolução, habilite o suporte a High DPI no Delphi moderno:

1. No arquivo `.dpr`, adicione:

```

// No arquivo .dpr:

begin
  Application.Initialize;
  Application.MainFormOnTaskbar := True;
  Application.HighDPIMode := True;    // Delphi 10.4+
  Application.CreateForm(TForm1, Form1);
  Application.Run;
end.

```

2. No manifesto (gerado automaticamente), defina `dpiAware = true` e `PerMonitorV2`.

No FireMonkey: o próprio FMX lida bem com High DPI usando a propriedade `ScaleFactor`. Você pode ajustar globalmente com `Application.ScaleManager`.

Dica 7 - Usar LiveBindings para sincronizar dados sem código (nova)

O LiveBindings permite conectar componentes visuais a fontes de dados (TFDQuery, TClientDataSet) sem escrever eventos OnGetText ou loops manuais.

Exemplo rápido: arraste um TListBox para o formulário, vincule um TBindSourceDB (conectado a uma query) e crie uma expressão de ligação como

```
ListBox1.SelectedItem = BindSourceDB1.FieldByName('nome').Value
```

Isto reduz drasticamente o código e torna a manutenção mais fácil. Vale estudar o LiveBindings Designer – uma das ferramentas mais poderosas e subestimadas do Delphi.

Dica 8 - Otimizar TFDQuery para grandes volumes de dados (FireDAC)

Ao trabalhar com tabelas de milhares de registros, evite travar a interface e o consumo excessivo de memória. Configure a *query* adequadamente:

```
with FDQuery1 do
begin
    FetchOptions.RecordCountMode := cmTotal;
    FetchOptions.Mode := fmOnDemand;      // carrega sob demanda
    FetchOptions.RowsLimit := 200;        // número de linhas por vez
    FetchOptions.Cache := [];             // não armazena em cache
    // Para edição em lote:
    CachedUpdates := True;
    // depois de alterar:
    ApplyUpdates;
    CommitUpdates;
end;
```

Isto é especialmente útil em aplicações multi-camadas (DataSnap, REST) ou com bancos remotos.

Conclusão

O Delphi não parou no tempo; as dicas que funcionavam há 15 anos ainda podem ser úteis – mas algumas são obsoletas e outras podem ser implementadas de forma mais elegante com os recursos atuais. A chave é conhecer a evolução da linguagem e da plataforma. Adote as boas práticas modernas: use FireDAC, LiveBindings, High DPI awareness, e evite truques que violem a segurança ou a experiência do usuário. O Delphi continua sendo uma ferramenta insuperável para aplicações desktop e, agora, também para soluções multiplataforma.
